

LLMs 101

Overview for today

- Components of our default training stack in more depth
- Code in our repository
- Personal workflow for experiments

Who this lecture is for

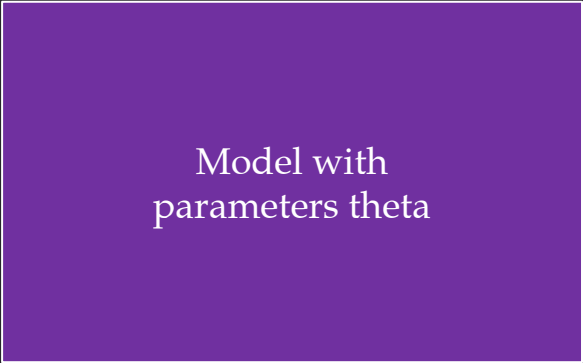
- Students who need to learn the “vocabulary” of deep learning
- Students who have trained models but don’t know what the internals look like

Who this lecture is not for

- Students who are deeply familiar with DL/LLM’s (e.g. taken 336)
- Students who are hoping to hear about exciting deep learning phenomenology or motivations (that’s every other lecture!)

Cartoon of pre-training

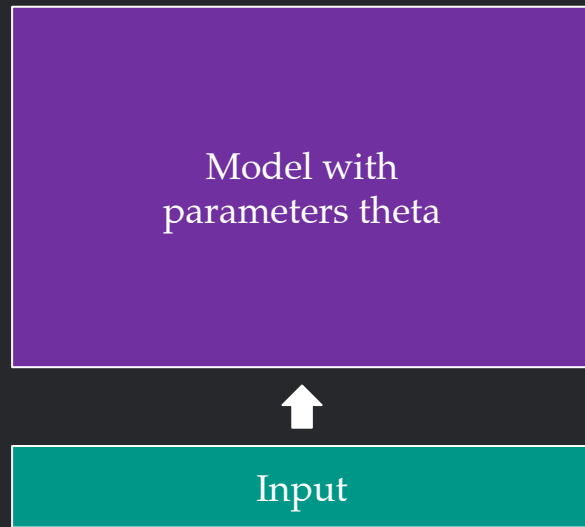
Four main parts when specifying a deep learning algorithm



Model with
parameters θ

Cartoon of pre-training

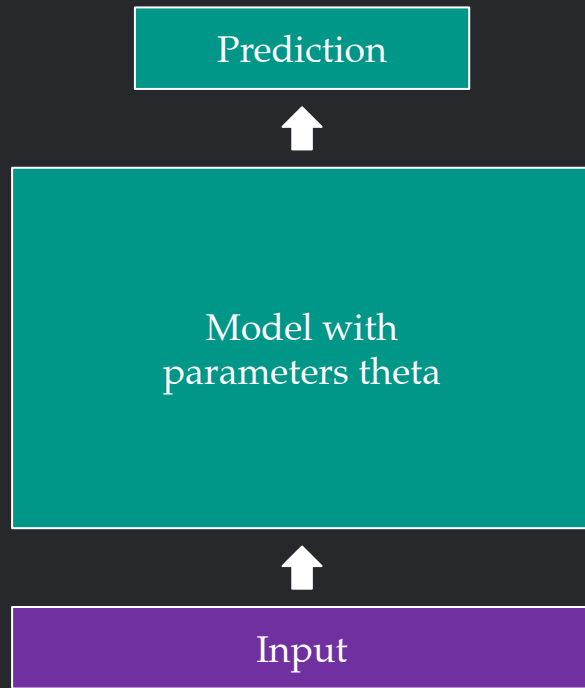
Data specifies a distribution over inputs to the model
(e.g. internet text)



Cartoon of pre-training

Data specifies a distribution over inputs to the model
(e.g. internet text)

Architecture specifies how θ maps to a model
prediction (e.g. transformer produces log likelihoods)

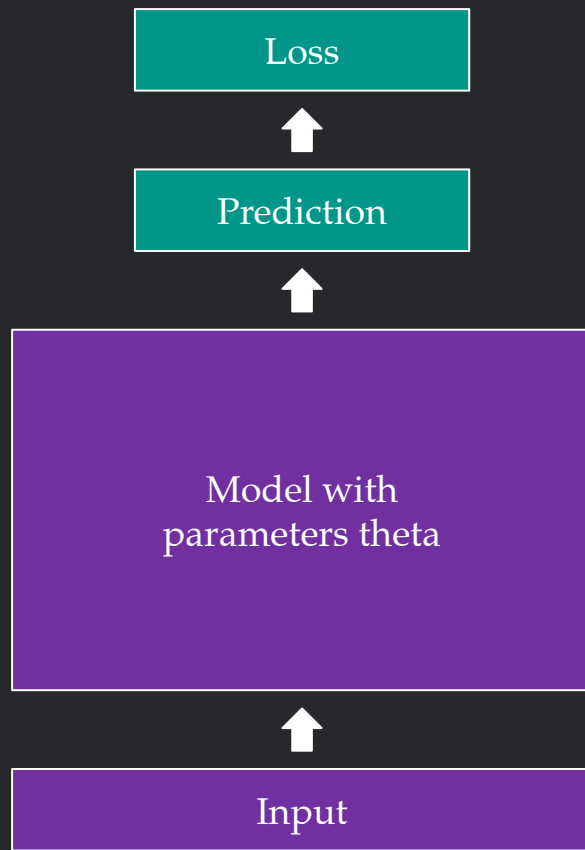


Cartoon of pre-training

Data specifies a distribution over inputs to the model
(e.g. internet text)

Architecture specifies how θ maps to a model
prediction (e.g. transformer produces log likelihoods)

Objective specifies error of θ via the loss function
(e.g. cross-entropy loss on predicted likelihoods)



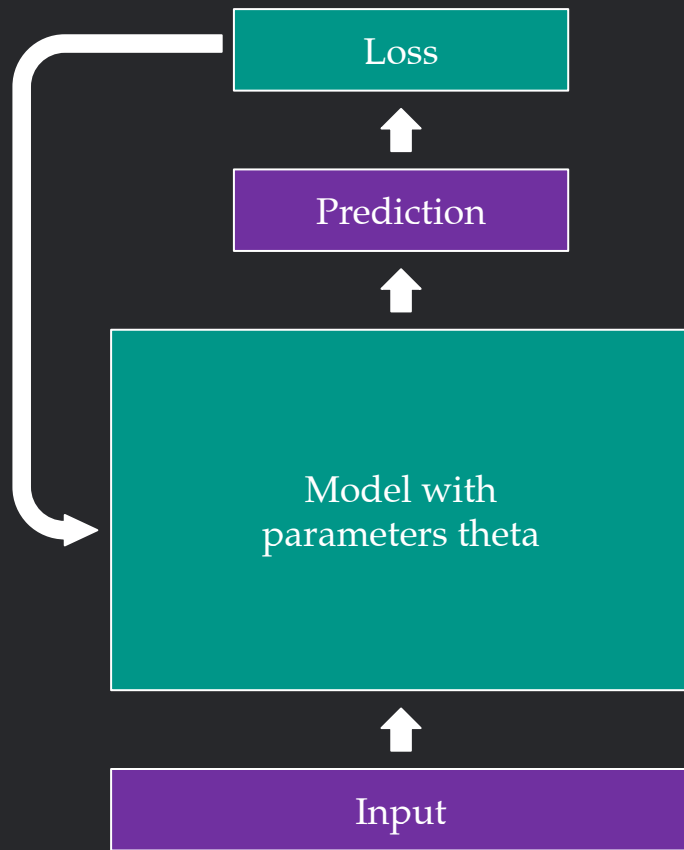
Cartoon of pre-training

Data specifies a distribution over inputs to the model (e.g. internet text)

Architecture specifies how θ maps to a model prediction (e.g. transformer produces log likelihoods)

Objective specifies error of θ via the loss function (e.g. cross-entropy loss on predicted likelihoods)

Optimization specifies how the model should update itself based on its loss (e.g. gradient descent on cross-entropy loss)



Default training stack

We'll go over a mix of general deep learning concepts as well as our training stack

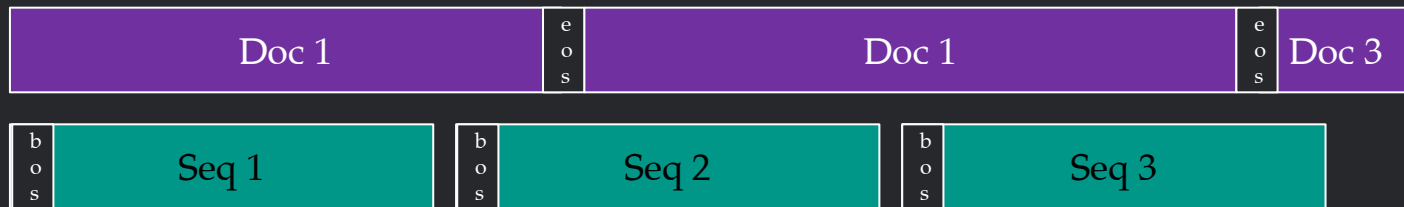
Our training stack is not perfect and definitely not state-of-the-art. However, it does reflect many important aspects of deep learning practice. At the minimum, you will need to be familiar with many components of it.

It is okay if you don't understand/memorize every single component (I definitely don't). We'll direct your attention to relevant components in assignments when we can

Data

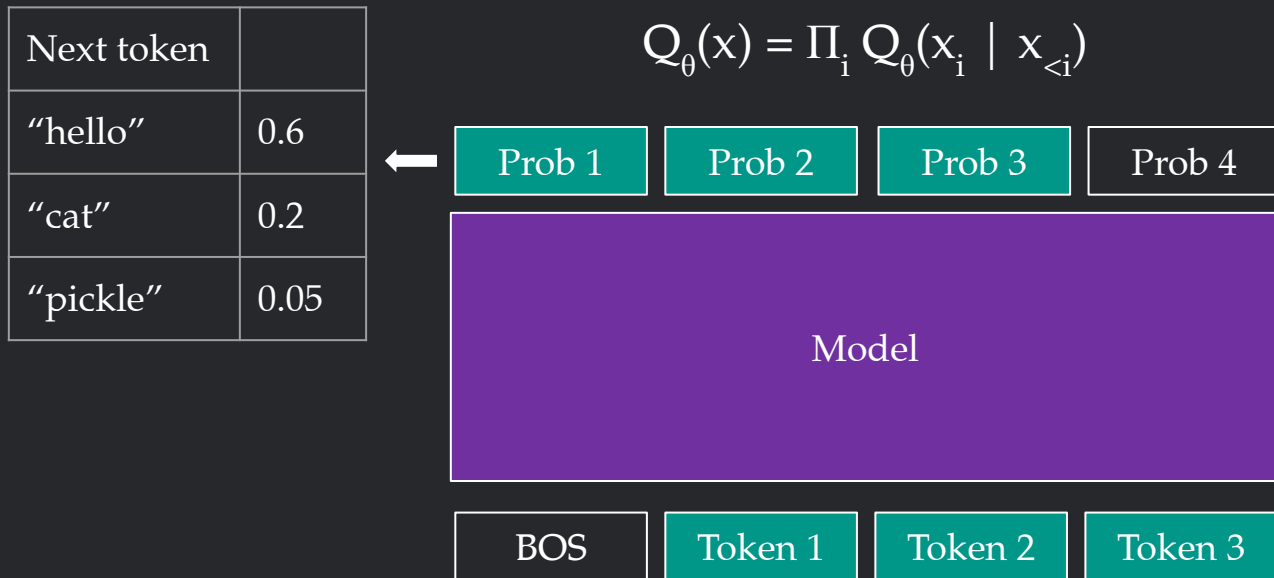
We collect internet documents, tokenize them with 4096 tokens, concatenate them

Our default setting has 600,000 sequences of 1024 tokens (1 beginning of sequence (BOS) token that is not predicted, 1023 actual tokens)



Default objective

Our model Q_θ predicts the likelihood of a sequence x from ground-truth distribution P . We will use auto-regressive models $Q_{\theta'}$ which represent



Default objective

We measure Q_θ 's error via cross-entropy loss $L(\theta)$, or the negative log likelihood assigned by Q_θ to real samples (often a held-out validation set)

$$L(\theta) = \mathbb{E}_{x \sim P}[-\log Q_\theta(x)] = \mathbb{E}_{x \sim P}[-\sum_i \log Q_\theta(x_i \mid x_{<i})]$$

We then normalize by the sequence length to measure loss per token. We want the loss to be as low as possible.

Next token	Prob
"hello"	0.6
"cat"	0.2
<u>"pickle"</u>	<u>0.05</u>
...	...



$$\log(1/0.05) = 4.605$$

Default objective

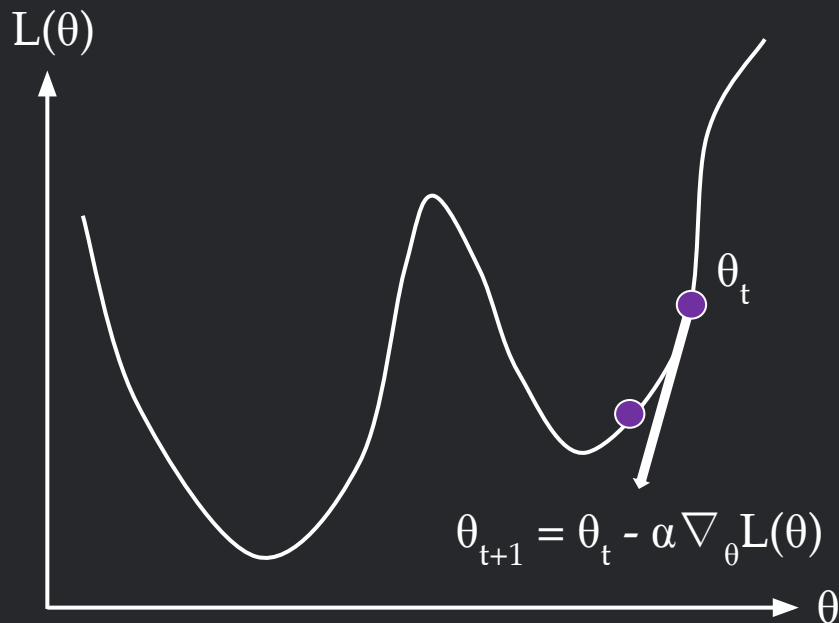
Code computing this in repo

```
def causal_lm_loss(logits, input_ids):  
    """Mean next-token negative log-likelihood for causal language modeling.  
    """  
  
    shift_logits = logits[..., :-1, :].contiguous().float()  
    shift_labels = input_ids[..., 1:].contiguous()  
    log_probs = F.log_softmax(shift_logits, dim=-1)  
    target_log_probs = log_probs.gather(  
        dim=-1,  
        index=shift_labels.unsqueeze(-1),  
    ).squeeze(-1)  
    return -target_log_probs.mean()
```

Optimization

We want to be able to move the function with a gradient update. However, estimating the gradient of the loss requires an expectation over all of the data

We estimate the gradient with a specified minibatch (64 by default)



Optimization

To update each model parameter, we need to compute $\nabla_{\theta} L(\theta)$. The core idea is to first compute the gradient for the last layer and then compute the gradient for every preceding layer.

$$h^{(\ell)} = f_{\ell}(h^{(\ell-1)}; \theta^{(\ell)})$$

$$\frac{\partial L}{\partial \theta_i^{(\ell)}} = \sum_j \frac{\partial L}{\partial h_j^{(\ell)}} \frac{\partial h_j^{(\ell)}}{\partial \theta_i^{(\ell)}},$$

gradient for each parameter,

$$\frac{\partial L}{\partial h_i^{(\ell-1)}} = \sum_j \frac{\partial L}{\partial h_j^{(\ell)}} \frac{\partial h_j^{(\ell)}}{\partial h_i^{(\ell-1)}},$$

gradient passed to the previous layer.

Optimization

Modern machine learning packages like PyTorch can automatically compute this gradient via autograd. Common workflow

```
input_ids = train_batches[batch_idx]
loss = training_loss(input_ids)
loss = loss / config.num_micro_batches
loss.backward()
optimizer.step()
```

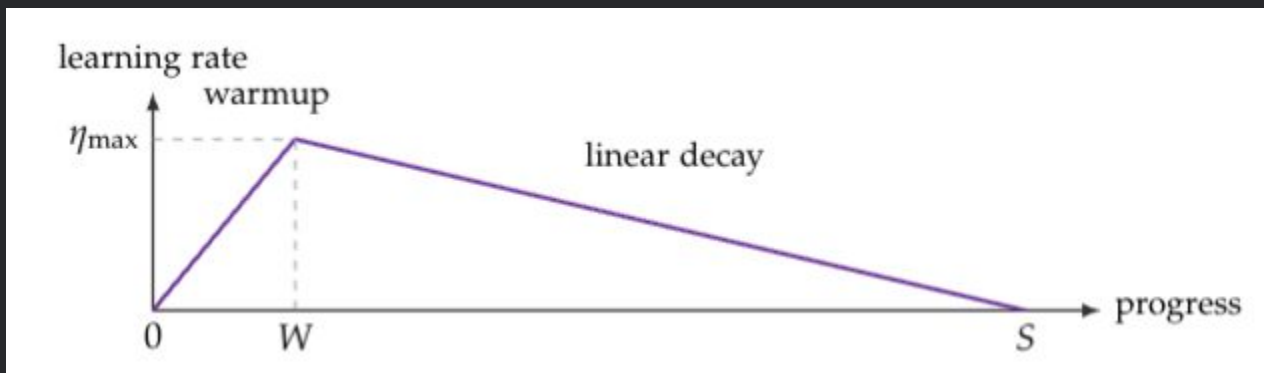
Optimization

Epochs are the number of passes over the training data. For most of our experiments until Unit 6, you can assume that we take one epoch over the training data

Optimization

Learning rate controls the size of the step. We schedule the learning rate over the course of training. By default, we use linear learning rate decay (warmup for first 1% of training, linearly decay for the rest) with peak learning rate 0.003

Alternatives include cosine decay and Warmup-Stable-Decay (flat after warmup and decay for the last x% of training)



Optimization

Momentum accumulates the direction from past gradients using an EMA. If the previous gradients are all pointing in the same direction, momentum accelerates progress in this direction. If the previous gradients disagree, momentum cancels out their contribution.

SGD with momentum (introduced hyperparameter μ):

$$b_s = \mu b_{s-1} + g_s, \quad \theta_{s+1} = \theta_s - \alpha_s b_s, \quad b_0 = 0,$$

Optimization

Adam extends SGD by also maintaining a moving average of the squared gradients (second moment). One potential reason to normalize the squared gradient is to take smaller steps for parameters that are more noisy

This optimizer now has two hyperparameters, beta1 (default 0.9) and beta2 (default 0.95)

$$m_s = \beta_1 m_{s-1} + (1 - \beta_1) g_s,$$

average gradient,

$$v_s = \beta_2 v_{s-1} + (1 - \beta_2) (g_s \odot g_s),$$

average squared gradient,

$$\hat{m}_s = m_s / (1 - \beta_1^s), \quad \hat{v}_s = v_s / (1 - \beta_2^s),$$

initialization-bias correction,

$$\theta_{s+1} = \theta_s - \alpha_s \frac{\hat{m}_s}{\sqrt{\hat{v}_s} + 10^{-8}},$$

update before weight decay.

Optimization

Weight decay shrinks a model's parameters towards the origin. There are many orthogonal intuitions for why it can be helpful (optimization stability, regularizing the model to be simpler, etc.). This introduces a new hyperparameter lambda, defaulted to 0.1

By default we will not apply weight decay on all the parameters (we will exclude embedding, normalization parameters, and bias)

$$\theta_{s+1} = (1 - \alpha_s \lambda) \theta_s - \alpha_s \frac{\hat{m}_s}{\sqrt{\hat{v}_s} + 10^{-8}}.$$

Optimization

Since certain gradients might be unusually large, we clip gradients that exceed a certain norm. In our setting, if the gradient norm exceeds 1, we rescale it to have norm 1 instead.

$$g_s^{\text{clip}} = g_s \min \left(1, \frac{c}{\|g_s\|_2} \right)$$

Optimization

There are certain diagnostics you can track to measure optimization health. We typically measure Root Mean Square (RMS) since this metric divides by the number of entries. Couple of different metrics you can track

- RMS of parameters
- RMS of activations
- RMS of gradients
- per module or globally

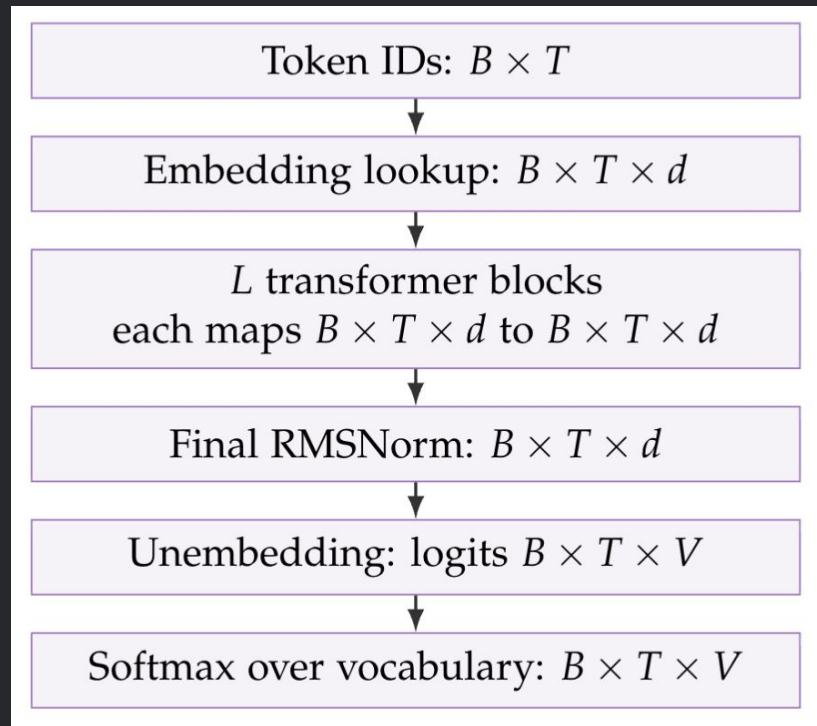
Architecture

Overview of architecture (batch size B , sequence length T , dimension d hidden)

Embeddings map tokens into d -dim vectors (hidden state, representations, activations, etc.)

Each block preserves the shape

Unembedding converts into next-token probabilities by mapping to V dimensions and applying softmax



$$p_{\theta}(x_{t+1} = v \mid x_{\leq t}) = \frac{e^{z_{t,v}}}{\sum_{u=0}^{V-1} e^{z_{t,u}}}$$

Embedding layer

Input $B \times T$ matrix of tokens, alternatively $B \times T \times V$ one-hot matrix of tokens

Output $B \times T \times d$ matrix of representations

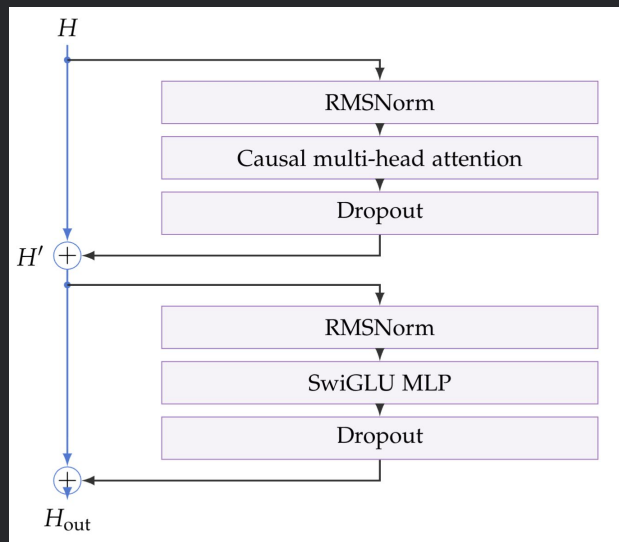
The embedding matrix is represented as a $V \times d$ matrix, associating each token with one d -dimensional representation. The output is equivalent to a matrix multiplication, but more efficiently implemented as a lookup table (due to one-hot matrix)

Layers/blocks

Overview of each of the blocks

Each block takes $B \times T \times d$ input and gives $B \times T \times d$ output

This allows us to stack as many layers as we'd like (we default to 8)



$$\begin{aligned} Y &= \text{RMSNorm}_{\text{attn}}(H), \\ A &= \text{Attention}(Y), \\ H' &= H + \text{Dropout}(A), \\ U &= \text{RMSNorm}_{\text{mlp}}(H'), \\ F &= \text{MLP}(U), \\ H_{out} &= H' + \text{Dropout}(F), \end{aligned}$$

$$\begin{aligned} Y &\in \mathbb{R}^{B \times T \times d}, \\ A &\in \mathbb{R}^{B \times T \times d}, \\ H' &\in \mathbb{R}^{B \times T \times d}, \\ U &\in \mathbb{R}^{B \times T \times d}, \\ F &\in \mathbb{R}^{B \times T \times d}, \\ H_{out} &\in \mathbb{R}^{B \times T \times d}. \end{aligned}$$

Residual connections

Instead of mapping $x \rightarrow f(x)$, a residual connection maps $x \rightarrow x + f(x)$

We apply residual connections to the attention layers and MLP layers

As you saw in the first lecture, it can be weirdly important to get this detail correctly

RMSNorm (Prenorm)

RMSNorm normalizes the hidden states to have a fixed norm and then rescales

Each entry of the vector is divided by the RMS (with a small constant for numerical stability). A subsequent learnable vector gamma controls the actual scale of the hidden state for each entry.

We apply this before attention and MLP, making it prenorm. If it was after, it'd be postnorm

$$\text{RMSNorm}(h)_i = \gamma_i \frac{h_i}{\sqrt{d^{-1} \sum_{j=1}^d h_j^2 + 10^{-5}}}, \quad i = 1, \dots, d.$$

Attention

- (1-3) produces queries, keys, values with dimension d_h
- (4-5) computes $T \times T$ self-attention matrix
- (6) applies self-attention matrix to mix value vectors
- (7) projects from d_h to d (in case they're not equal)

$$Q = YW_Q \in \mathbb{R}^{T \times d_h},$$

queries,

$$K = YW_K \in \mathbb{R}^{T \times d_h},$$

keys,

$$V_a = YW_V \in \mathbb{R}^{T \times d_h},$$

values,

$$S = QK^\top / \sqrt{d_h} + C \in \mathbb{R}^{T \times T},$$

masked similarity scores,

$$P_a = \text{softmax}_{\text{keys}}(S),$$

weights over source positions,

$$O = P_a V_a \in \mathbb{R}^{T \times d_h},$$

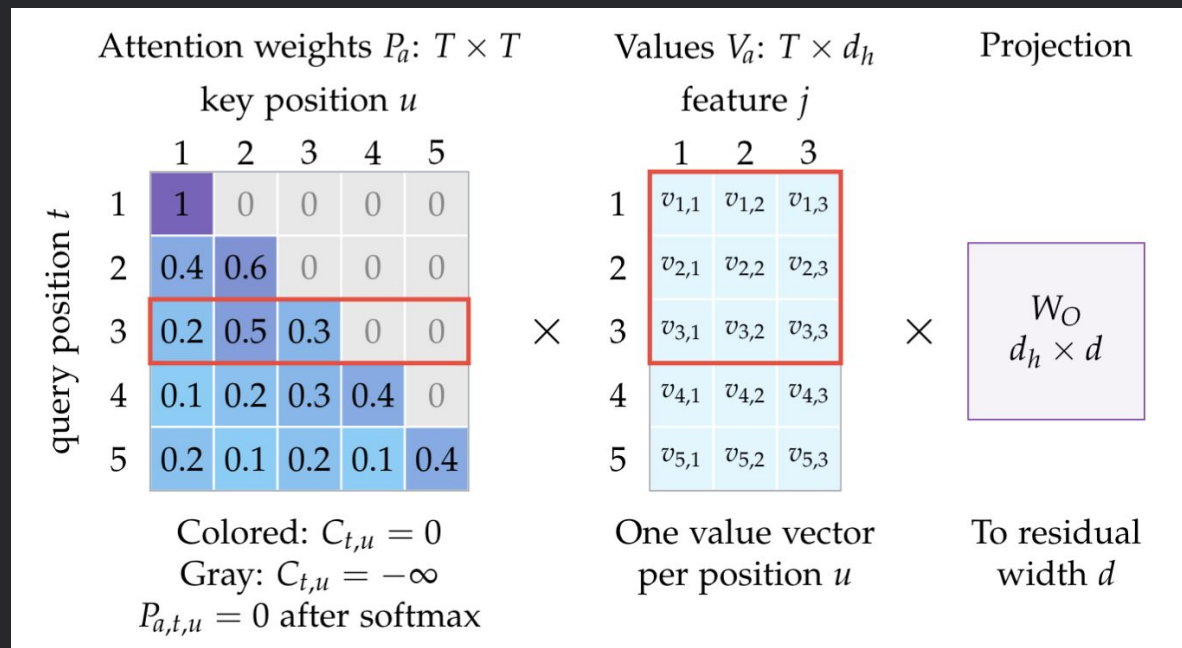
weighted mixtures of values,

$$\text{Attention}(Y) = OW_O \in \mathbb{R}^{T \times d},$$

project back to residual-stream width.

Attention

P_a corresponds to the attention matrix, $T \times T$ giving similarity between past tokens and future tokens



Multi-head attention

Multi-head attention implements H heads to be run in parallel for a total of Hd_h dimension for queries, keys, and values (efficient implementation in code)

By default, we set $d_h = 64$, $H = 8$, $d = 512$. When d increases, we default to increasing H while keeping d_h fixed

$$O^{(h)} = \text{Head}_h(Y) \in \mathbb{R}^{T \times d_h},$$

$$h = 1, \dots, H,$$

$$O_{\text{concat}} = [O^{(1)} \ \dots \ O^{(H)}] \in \mathbb{R}^{T \times Hd_h},$$

concatenate head outputs,

$$\text{Attention}(Y) = O_{\text{concat}} W_O \in \mathbb{R}^{T \times d},$$

return to residual-stream width.

Positional embeddings (RoPE)

Other than the causal mask, there is no actual indication about the positions of the tokens. RoPE encodes the relative positions by rotating pairs of dimensions in the query and pairs of dimensions in the keys

$$R(t\omega_i) = \begin{pmatrix} \cos(t\omega_i) & -\sin(t\omega_i) \\ \sin(t\omega_i) & \cos(t\omega_i) \end{pmatrix}, \quad \omega_i = \theta_{\text{RoPE}}^{-2i/d_h}, \quad i = 0, \dots, d_h/2 - 1.$$

$$\begin{pmatrix} \tilde{q}_{t,i} \\ \tilde{q}_{t,i+d_h/2} \end{pmatrix} = R(t\omega_i) \begin{pmatrix} q_{t,i} \\ q_{t,i+d_h/2} \end{pmatrix},$$

rotate each query pair,

$$\begin{pmatrix} \tilde{k}_{t,i} \\ \tilde{k}_{t,i+d_h/2} \end{pmatrix} = R(t\omega_i) \begin{pmatrix} k_{t,i} \\ k_{t,i+d_h/2} \end{pmatrix},$$

rotate each key pair.

Positional embeddings (RoPE)

The important quantity is that when forming Q^TK , we get an inner product which depends only on $R(t\omega_i)^TR(u\omega_i) = R((u - t)\omega_i)$ which only depends on the offset. This yields relative positional embeddings

$$R(t\omega_i) = \begin{pmatrix} \cos(t\omega_i) & -\sin(t\omega_i) \\ \sin(t\omega_i) & \cos(t\omega_i) \end{pmatrix}, \quad \omega_i = \theta_{\text{RoPE}}^{-2i/d_h}, \quad i = 0, \dots, d_h/2 - 1.$$

$$\begin{pmatrix} \tilde{q}_{t,i} \\ \tilde{q}_{t,i+d_h/2} \end{pmatrix} = R(t\omega_i) \begin{pmatrix} q_{t,i} \\ q_{t,i+d_h/2} \end{pmatrix}, \quad \text{rotate each query pair,}$$
$$\begin{pmatrix} \tilde{k}_{t,i} \\ \tilde{k}_{t,i+d_h/2} \end{pmatrix} = R(t\omega_i) \begin{pmatrix} k_{t,i} \\ k_{t,i+d_h/2} \end{pmatrix}, \quad \text{rotate each key pair.}$$

QK Norm

Another stability trick is to normalize the queries and keys. Before computing the attention matrix, we simply apply an RMS Norm to the queries and keys.

MLP

The MLP is itself a multilayer network that applies a nonlinear computation on a per-token basis. The MLP we use is implemented as a SwiGLU: one linear layer up-projecting to dimension d_{ff} , one non-linearity implemented via SiLU with a gate, and one additional non-linearity.

$$G = YW_g, \quad U = YW_u,$$

$$A = \text{SiLU}(G) \odot U,$$

$$\text{MLP}(Y) = AW_d,$$

$$G, U \in \mathbb{R}^{B \times T \times d_{\text{ff}}},$$

$$A \in \mathbb{R}^{B \times T \times d_{\text{ff}}},$$

$$\text{MLP}(Y) \in \mathbb{R}^{B \times T \times d}.$$

Unembedding

The unembedding layer maps $B \times T \times d$ representations to a vector $B \times T \times V$. This final output is interpreted as the log-probabilities predicted by the model (up to a constant that would make a valid probability distribution) To translate the log-probabilities into probabilities, we apply a softmax

Softmax is translation invariances

$$p_{\theta}(x_{t+1} = v \mid x_{\leq t}) = \frac{e^{z_{t,v}}}{\sum_{u=0}^{V-1} e^{z_{t,u}}}.$$

Dropout

Dropout randomly masks a fraction p of the entries of the activation (and rescales other entries by $1/p$). One intuition is that it forces the model to learn distributed representations and multiple ways of fitting the data

During evaluation time, dropout is disabled

Initialization

We initialize parameters depending on the dimension of the neural network. Our current implementation uses the following schematic

- Each parameter (except for RMSNorm scales) is drawn from a mean-zero Gaussian truncated to $[-3, 3]$ and then further rescaled
- Attention and MLP matrices are scaled to standard deviation $1/\sqrt{d_{\text{in}}}$
- Embedding is $1/d$ std and unembedding is $1/\sqrt{d}$ std
- Embedding and unembedding are both the same Gaussian scaled differently (so they point in the same direction)

Scaling

We provide a default recipe for training larger models

- Provide the depth of the model (number of blocks) (i.e. $L = 8$)
- Provide residual width (i.e. $d = 64L$)
- Provide number of heads and head dimension (i.e. $H = L$, $d_h = 64$)
- MLP width (i.e. $d_{\text{ff}} = 3.5d$)
- Initialization variance (specified last slide)

Because of our choice of scaling recipe, attention and MLP parameters grow roughly d^3 . If larger models are trained on more tokens, that's a lot of compute!

Our code and runs

To the code!